

Difference Between Transformations and Actions in Apache Spark

Spark operations are divided into two categories:

1. **Transformations**
2. **Actions**

The key difference is:

- **Transformations** create a new DataFrame or RDD **without executing** the computation immediately.
 - **Actions** trigger the execution of all pending transformations and either return a result or write data to storage.
-

What is a Transformation?

A **Transformation** is an operation that creates a **new DataFrame or RDD** from an existing one.

Transformations are **lazy**, meaning Spark **does not execute them immediately**. Instead, Spark records the operations in a **lineage (execution plan)**.

Characteristics

- Lazy evaluation
- Returns a new DataFrame or RDD
- Does not execute immediately
- Builds the logical execution plan (DAG)
- Can be chained together

Example

```
df_filtered = df.filter(df.salary > 5000)

df_selected = df_filtered.select("name", "salary")
```

At this point:

- No data has been read.
- No computation has been performed.
- Spark has only recorded the transformations.

What is an Action?

An **Action** is an operation that **triggers the execution** of all previously defined transformations.

Actions either:

- Return a result to the Driver, or
- Save data to external storage.

Characteristics

- Triggers job execution
- Produces a result
- Starts the DAG execution
- Causes Spark to read and process the data

Example

```
df_filtered.show()
```

When `show()` is called:

- Spark executes all pending transformations.
- Reads the data.
- Processes it across executors.
- Returns the first rows to the Driver.

Example Workflow

```
df = spark.read.csv("employees.csv", header=True)
```

```
df2 = df.filter(df.salary > 5000)
```

```
df3 = df2.select("name", "salary")
```

```
df3.show()
```

Execution flow:

Read CSV	→ Transformation
Filter	→ Transformation
Select	→ Transformation

Show → Action (execution starts)

Spark waits until `show()` is called before executing the entire pipeline.

Lazy Evaluation

Spark uses **lazy evaluation** to optimize execution.

Instead of executing each transformation immediately, Spark:

1. Records all transformations.
2. Builds a logical plan.
3. Optimizes the plan.
4. Executes everything only when an action is called.

This reduces unnecessary work and improves performance.

Common Transformations

DataFrame Transformations

- `select()`
- `filter()` / `where()`
- `withColumn()`
- `drop()`
- `dropDuplicates()`
- `distinct()`
- `groupBy()`
- `orderBy()` / `sort()`
- `join()`
- `union()`
- `limit()`
- `repartition()`
- `coalesce()`

RDD Transformations

- `map()`
- `flatMap()`
- `filter()`

- `distinct()`
- `union()`
- `intersection()`
- `reduceByKey()`
- `groupByKey()`
- `sortByKey()`

Common Actions

DataFrame Actions

- `show()`
- `collect()`
- `count()`
- `first()`
- `head()`
- `take()`
- `foreach()`
- `write.save()`
- `write.csv()`
- `write.parquet()`
- `write.json()`

RDD Actions

- `collect()`
- `count()`
- `first()`
- `take()`
- `reduce()`
- `foreach()`
- `saveAsTextFile()`

Comparison Table

Feature	Transformation	Action
Purpose	Define how data should be processed	Execute the processing
Execution	Lazy (not executed immediately)	Immediate (triggers execution)
Returns	New DataFrame or RDD	Result to the Driver or writes data

Feature	Transformation	Action
Builds DAG	Yes	Executes the DAG
Creates Spark Job	No	Yes

Example

```
df = spark.read.csv("employees.csv")

df = df.filter(df.salary > 5000)      # Transformation

df = df.select("name", "salary")     # Transformation

df.count()                           # Action
```

Execution order:

```
filter()
  ↓
select()
  ↓
count() ← Spark starts execution here
```

Why Does Spark Use Lazy Evaluation?

Lazy evaluation allows Spark to:

- Optimize the execution plan.
 - Combine multiple transformations into a single optimized pipeline.
 - Minimize data movement (shuffle).
 - Avoid unnecessary computations.
 - Improve overall performance.
-

Interview Questions

Q1. What is the difference between a transformation and an action?

- **Transformation:** Defines a new DataFrame or RDD and is evaluated lazily. It does not execute immediately.
- **Action:** Triggers the execution of all pending transformations and either returns a result or writes data.

Q2. Does a transformation execute immediately?

No. Transformations are lazy. Spark records them in the execution plan and waits until an action is called.

Q3. Which operation starts a Spark job?

An **Action** starts a Spark job by triggering the execution of the DAG.

Q4. Give examples of transformations.

Examples include:

- `filter()`
 - `select()`
 - `join()`
 - `groupBy()`
 - `withColumn()`
 - `repartition()`
 - `map()` (RDD)
-

Q5. Give examples of actions.

Examples include:

- `show()`
 - `count()`
 - `collect()`
 - `first()`
 - `take()`
 - `write.parquet()`
 - `saveAsTextFile()`
-

Interview Summary

- **Transformations** define how data should be processed. They are **lazy**, return a new DataFrame or RDD, and build Spark's execution plan without performing any computation.
- **Actions** trigger the execution of all pending transformations. They either return results to the Driver (e.g., `count()`, `collect()`, `show()`) or write data to storage (e.g., `write.parquet()`).
- Spark's **lazy evaluation** enables the Catalyst optimizer and scheduler to optimize the execution plan before running it, improving efficiency and reducing unnecessary computation.